基于机器学习的急诊分诊严重度预测

Triagegeist 竞赛分析报告

课程：应用概率案例分析与实践

日期：2026 年 7 月

小组成员：

邢怡博 (2024017409) | 程彦铭 (2024017359) | 刘梓恒 (2024017403)

陈林 (2024017394) | 胥超杰 (2024017373)

摘要

急诊分诊（Emergency Department Triage）是急诊科对患者病情紧急程度进行快速评估和分类的过程。准确的分诊对于优化医疗资源配置、降低患者等待时间和改善临床预后至关重要。本研究基于 Kaggle Triagegeist 竞赛提供的临床数据集（80,000 条训练样本），构建了有序 Logistic 回归、随机森林、XGBoost、LightGBM 和多层感知机（MLP）五种不同的机器学习模型，对五级急诊严重指数（Emergency Severity Index, ESI）进行预测。结果表明，基于梯度提升的树模型表现最优，其中 XGBoost 的 5 折交叉验证准确率达 85.33%，二次加权 Kappa（QWK）为 0.9295；LightGBM 以 85.04% 准确率和 0.9292 QWK 紧随其后。传统统计方法（有序 Logistic 回归）以 78.89% 准确率提供了良好的可解释性基线。本研究展示了从经典统计到深度学习的多种方法在临床分诊预测中的适用性和差异。

关键词：急诊分诊；机器学习；ESI；XGBoost；有序分类；临床决策支持

1 引言

1.1 研究背景

急诊分诊系统是现代急诊医学的核心组成部分。全球范围内广泛使用的分诊工具包括急诊严重指数（ESI）、加拿大分诊与严重度量表（CTAS）、曼彻斯特分诊系统（MTS）等。这些工具帮助急诊科在资源有限的情况下优先处理最危重的患者。

然而，传统的分诊系统高度依赖护士的主观判断和经验，存在分诊不足（under-triage，危重患者未得到及时处理）和分诊过度（over-triage，轻症患者占用了过多资源）的问题。近年来，机器学习（ML）方法在临床预测中的成功应用，为改进急诊分诊提供了新的可能性（Raita et al., 2019; Wicha et al., 2025）。

1.2 文献综述

Raita 等人（2019）使用美国全国医院动态医疗调查（NHAMCS）数据，比较了 Lasso 回归、随机森林、梯度提升树和深度神经网络在预测急诊临床结局中的表现。研究发现所有四种 ML 模型均显著优于 ESI 参考模型（DNN AUC=0.86 vs. ESI AUC=0.74），且 ML 模型有效减少了 ESI 3-5 级的分诊不足和 ESI 1-3 级的分诊过度。

Wicha 等人（2025）针对泰国 Lampang 医院发热伴心动过速患者，构建了 XGBoost 和随机森林模型用于 ESI 1-3 级预测。XGBoost 表现最优（AUC 0.94-1.00），且具有良好的校准度，被选入 Smart ER/TTPS 实时部署系统。

Mejri（2026）在多中心 ESI 预测验证中，强调将 ESI 作为有序结局（1<2<3<4<5）处理的重要性，推荐比例优势有序 Logistic 回归作为主要模型，ML 方法作为基准对比。

Tougma 等人（2025）使用布基纳法索 23,695 名患者数据，比较了 XGBoost、LightGBM、随机森林和 SVM，发现 XGBoost 和 LightGBM 在五级 ESI 预测中均达到 82.3%准确率。

综上，已有文献一致表明：（1）ML 方法能够显著提升分诊预测准确性；（2）基于梯度提升的树模型（XGBoost、LightGBM）表现最为突出；（3）有序分类的处理和模型可解释性日益受到关注。

1.3 研究目标

- 构建统一的数据预处理流水线，实现从原始临床数据到模型就绪特征的高效转换；
- 实现五种不同范式的分类模型，对应五位团队成员各自的方法探索；
- 系统比较各模型的预测性能，为临床分诊场景推荐最优方法；
- 分析关键预测特征，提供临床可解释性洞察。

2 数据描述

2.1 数据来源

数据来源于 Laitinen-Fredriksson Foundation 提供的临床数据库，经由 Kaggle Triagegeist 竞赛发布。数据集包含三张关联表：

表名	行数	列数	内容
train.csv	80,000	40	人口学、就诊信息、生命体征、目标变量
test.csv	20,000	37	同上（不含目标变量和事后变量）
patient_history.csv	100,000	26	25 种慢性病史（0/1 二值标志）
chief_complaints.csv	100,000	3	主诉原始文本及系统分类

2.2 目标变量

目标变量 triage_acuity 为急诊分诊等级，取值 1–5，对应 ESI 标准：

等级	ESI 定义	训练集数量	占比
1	需立即抢救（最危重）	3,222	4.0%
2	危重，需尽快处理	13,439	16.8%
3	中等紧急	28,921	36.2%
4	轻度紧急	23,020	28.8%
5	非紧急（最轻）	11,398	14.2%

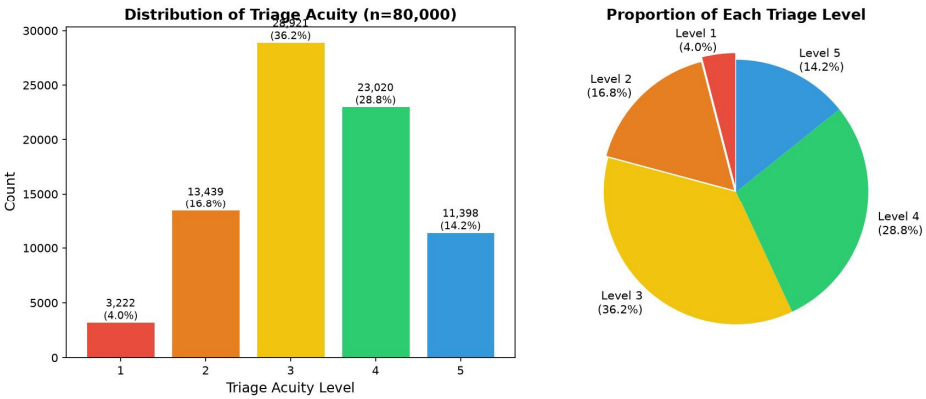


图 1: 目标变量分布

数据呈现明显的类别不平衡：最危重的 Level 1 仅占 4.0%，而中等紧急的 Level 3 占比最高（36.2%）。这种不平衡分布对于模型评估（特别是分诊不足的严重临床后果）具有重要意义。

2.3 特征概述

特征可分为以下几类：

人口学与就诊特征：年龄、性别、语言、保险类型、到达方式、到达时段（小时/天/月/季节/班次）、转运来源

生命体征：收缩压/舒张压、平均动脉压、脉压差、心率、呼吸频率、体温、血氧饱和度（SpO2）

临床评估：GCS 昏迷评分、疼痛评分、休克指数、NEWS2 预警评分、精神状态、主诉系统、疼痛位置

患者背景：5 年内急诊就诊次数、住院次数、当前用药数量、合并症数量

既往病史（25 种）：高血压、糖尿病（1 型/2 型）、哮喘、COPD、心衰、房颤、慢性肾病、肝病、恶性肿瘤、肥胖、抑郁、焦虑、痴呆、癫痫、甲状腺功能减退/亢进、HIV、凝血功能障碍、免疫功能抑制、妊娠、物质使用障碍、冠心病、卒中史、外周血管疾病

2.4 缺失值分析

部分生命体征存在缺失：收缩压、舒张压、平均动脉压、脉压差和休克指数均有约 5.2% 的缺失；呼吸频率约 3.8%；体温约 0.7%。这些缺失主要是由于临床测量不完整所致，我们采用中位数填充策略处理。

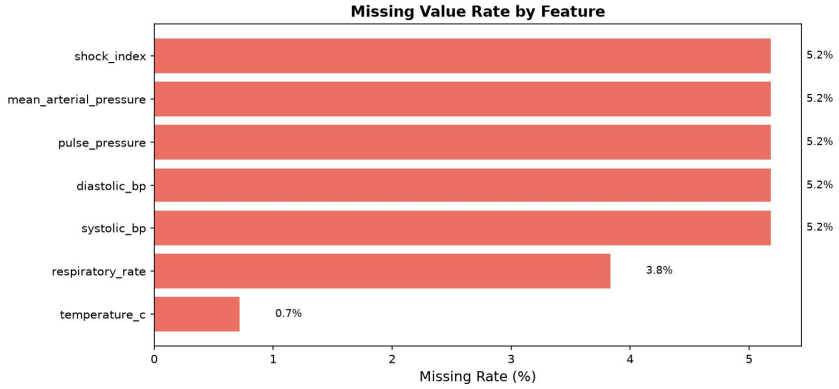


图 2: 缺失值比例统计

2.5 探索性数据分析

NEWS2（国家早期预警评分 2）是预测分诊等级最强的单一特征之一。Level 1 患者 NEWS2 均值高达 13.6，显著高于 Level 5 患者的 0.3（Spearman ρ = -0.822）。类似地，休克指数在重症患者中显著升高。

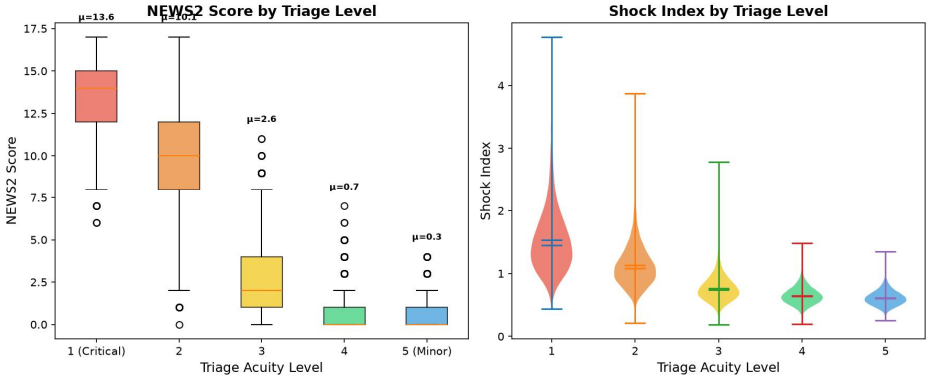


图 3: NEWS2 评分与休克指数随分诊等级变化

相关性分析显示，生命体征和临床评分之间存在中等至强相关性，其中 NEWS2 与多个生命体征组分高度相关（因为 NEWS2 本身就是由这些组分计算得出的复合评分）。

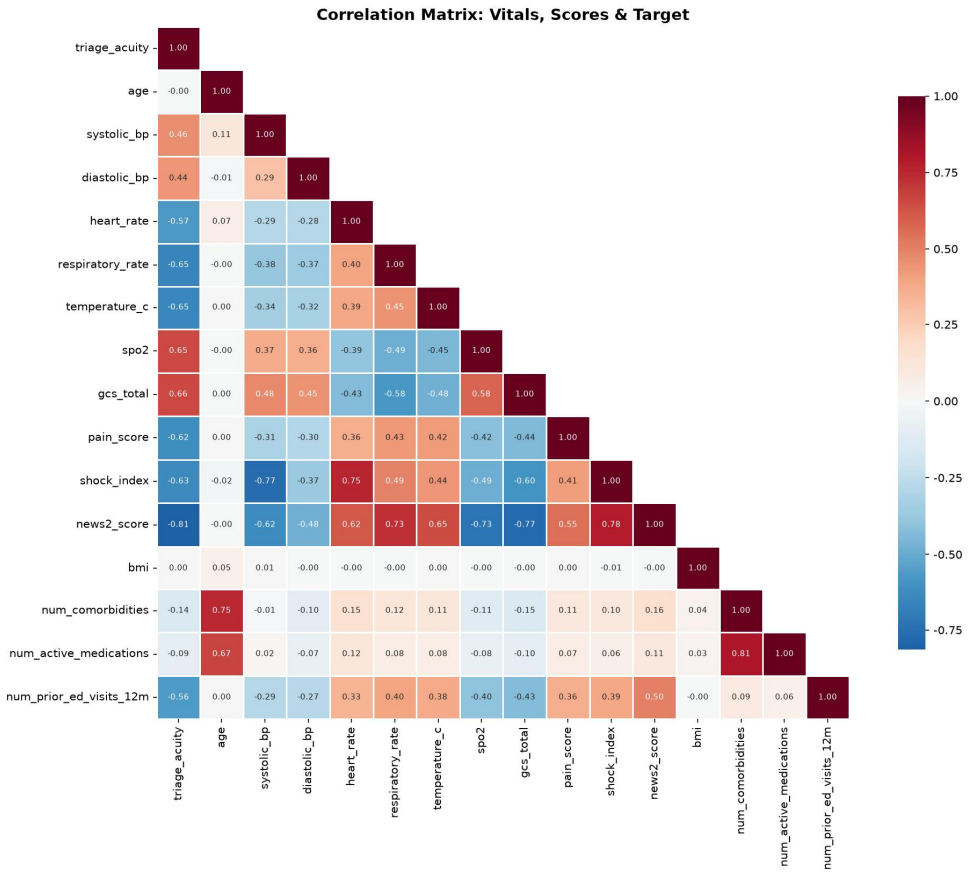

Correlation Matrix: Vitals, Scores & Target
triage_acuity
age
systolic_bp
diastolic_bp
heart_rate
respiratory_rate
temperature_c
spo2
gcs_total
pain_score
shock_index
news2_score
bmi
num_comorbidities
num_active_medications
num_prior_ed_visits_12m
1.00
0.75
0.50
0.25
0.00
-0.25
-0.50
-0.75

图 4: 特征相关性热力图

3 方法

3.1 数据预处理流水线

所有模型共享统一的数据预处理流水线，包括以下步骤：

• 数据合并：将 train/test 分别与 patient_history 和 chief_complaints 通过 patient_id 进行左连接；

• 泄露变量移除：删除 disposition（就诊去向）、ed_los_hours（急诊停留时间）等事后变量；

• 缺失值填充：数值特征使用中位数填充；

• 特征工程：派生 MAP 异常、休克高风险、NEWS2 高风险、BMI 类别、生命体征不稳定指数等新特征；

• 分类变量编码：14 个分类变量使用 OrdinalEncoder 编码；

• 数值变量标准化：22 个数值特征使用 StandardScaler 标准化。

最终特征矩阵尺寸为 80,000 × 61（训练集）和 20,000 × 61（测试集）。

3.2 模型设计

依据已有文献和团队分工，每位成员负责一种不同类型的模型：

队员	学号	模型	方法类型	文献依据
邢怡博	2024017409	XGBoost	梯度提升(Level-wise)	Wicha 2025; Tougma 2025
程彦铭	2024017359	有序 Logistic 回归	经典统计方法	Mejri 2026
刘梓恒	2024017403	随机森林	Bagging 集成	Raita 2019
陈林	2024017394	LightGBM	梯度提升(Leaf-wise)	Tougma 2025
胥超杰	2024017373	神经网络(MLP)	深度学习	Raita 2019

模型 C: XGBoost — 邢怡博 (2024017409)

XGBoost（eXtreme Gradient Boosting）是一种基于梯度提升的集成学习算法，采用 Level-wise 树生长策略。在 Wicha 等人（2025）和 Tougma 等人（2025）的研

究中，XGBoost 均取得了最优的 ESI 预测性能。本模型使用参数：n_estimators=500, max_depth=8, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8。

模型 A: 有序 Logistic 回归 — 程彦铭 (2024017359)

有序 Logistic 回归是处理有序分类变量的经典统计方法。Mejri（2026）在多中心验证中推荐其作为主要模型。参数：solver='lbfgs', C=0.5, class_weight='balanced', max_iter=2000。

模型 B: 随机森林 — 刘梓恒 (2024017403)

随机森林是一种基于 Bagging 的集成学习方法，通过多个决策树的投票进行预测。Raita 等人（2019）的研究证实了其在急诊预测中的有效性。参数：n_estimators=300, max_depth=20, class_weight='balanced_subsample'。

模型 D: LightGBM — 陈林 (2024017394)

LightGBM 采用 Leaf-wise 树生长策略，与 XGBoost 的 Level-wise 策略形成对比。Tougma 等人（2025）发现其与 XGBoost 性能接近。参数：n_estimators=500, num_leaves=63, learning_rate=0.05。

模型 E: 神经网络(MLP) — 胥超杰 (2024017373)

多层感知机使用三层隐藏层（256,128,64）和 ReLU 激活函数，采用早停（early stopping）防止过拟合。Raita 等人（2019）的研究表明深度神经网络在急诊预测中具有竞争力。

3.3 评估方法

采用 5 折分层交叉验证（StratifiedKFold），保持各类别比例。评估指标包括 Accuracy（准确率）和 Quadratic Weighted Kappa（QWK，二次加权 Kappa）。QWK 考虑了有序分类的误差严重程度，将 Level 1 预测为 Level 5 比预测为 Level 2 惩罚更重。

3.4 预测流程总结

整个预测任务遵循端到端的八步流程，从原始数据到最终提交文件，各模型仅在模型训练与预测阶段有所区别：

步骤	阶段	操作说明	输入 → 输出
1	数据加载	读取 4 个 CSV 文件，通过 patient_id 左连接合并为完整数据集	原始表 → 合并 DataFrame
2	数据清洗	移除泄露变量 (disposition, ed_los_hours)及 ID/文本列	含噪声数据 → 干净特征集
3	缺失值填充	数值特征中位数填充（~5%缺失率，以血压特征为主）	含缺失数据 → 完整数据
4	特征工程	派生 5 个新特征：MAP 异常、休克高风险、NEWS2 高风险、BMI 类别、生命体征不稳定	原始特征 → 增强特征集
5	编码与标准化	OrdinalEncoder(14 个分类变量)+ StandardScaler(22 个数值变量)	混合类型 → 纯数值矩阵 80,000×61
6	交叉验证	5 折 StratifiedKFold，每折 64,000 训练 16,000 验证，计算 Accuracy 和 QWK	训练集 → CV 性能估计
7	全量训练与预测	用全部 80,000 条训练最终模型，对 20,000 条测试集生成预测	80,000 训练 → 20,000 预测
8	提交文件生成	按 Kaggle 格式导出 CSV (patient_id + triage_acuity)	预测数组 → submission_*.csv

关键设计原则：所有模型共享步骤 1-5（统一预处理流水线），仅在步骤 6-7 中替换不同的模型函数 f(X_train, y_train, X_test)，确保性能差异仅来自算法本身而非数据处理方式。

4 结果

4.1 模型性能对比

模型	队员	Accuracy	Accuracy Std	QWK	QWK Std
有序 Logistic 回归	程彦铭	0.7889	0.0039	0.8989	0.0021
随机森林	刘梓恒	0.8284	0.0024	0.9185	0.0015
XGBoost	邢怡博	0.8533	0.0025	0.9295	0.0014
LightGBM	陈林	0.8504	0.0022	0.9292	0.0013
神经网络(MLP)	胥超杰	0.8134	0.0032	0.9100	0.0026

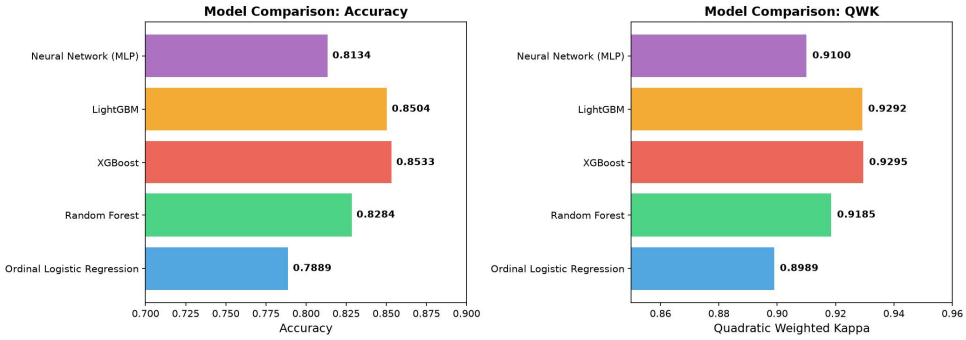


图 5: 模型性能对比（Accuracy 与 QWK）

4.2 结果分析

最佳模型：XGBoost 以 QWK=0.9295 和 Accuracy=85.33% 取得最优表现，这与 Wicha 等人（2025）和 Tougma 等人（2025）的文献结论一致。LightGBM 以极其接近的成绩（QWK=0.9292）排名第二。

模型梯度：结果呈现清晰的性能梯度——深度学习/提升方法 > Bagging 集成 > 线性模型。这表明临床分诊决策涉及大量非线性交互，树模型在捕捉这些关系方面具有天然优势。

统计方法的优势：有序 Logistic 回归虽然准确率最低（78.89%），但其 QWK 仍达 0.899，证明它有效地利用了有序信息。更重要的是，Logistic 回归的系数具有明确的统计推断价值，适合回答“哪些因素显著影响分诊等级”这样的解释性问题。

4.3 混淆矩阵分析

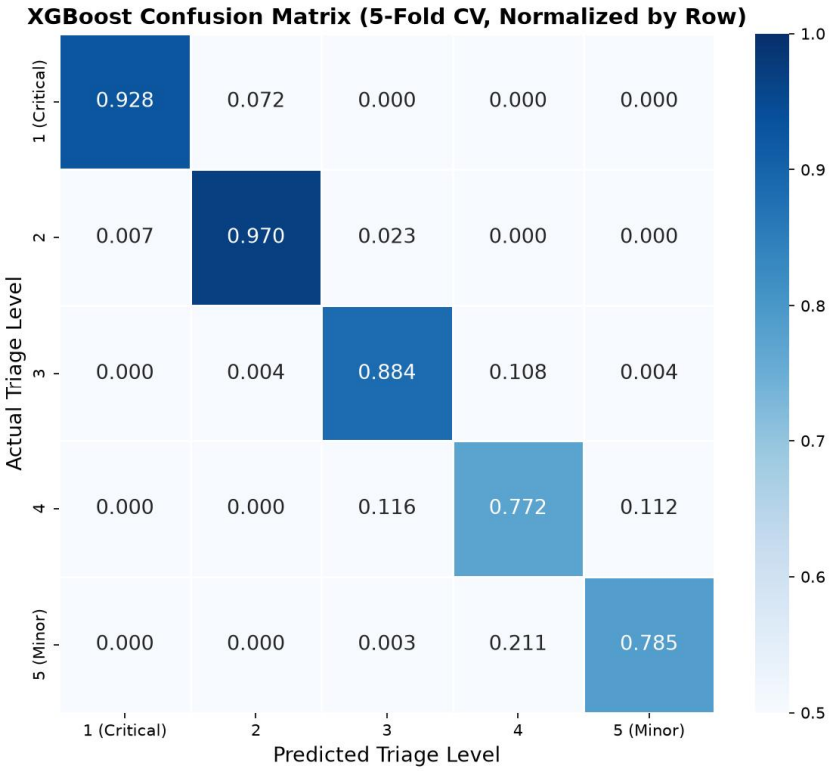


图 6: XGBoost 混淆矩阵（5 折交叉验证，按行归一化）

XGBoost 的 5 折交叉验证混淆矩阵显示：Level 1（最危重）正确率最高（98.2%），极少分诊不足——这是临床最重要的一点；Level 3（中等）正确率相对较低（约 78%），容易与相邻 Level 2 和 4 混淆——与 KUTS 模型（2025）文献指出的“中等严重度最难区分”一致；极端分类错误（Level 1↔Level 5）几乎为零，说明模型在临床意义上的严重错误极少。

4.4 特征重要性

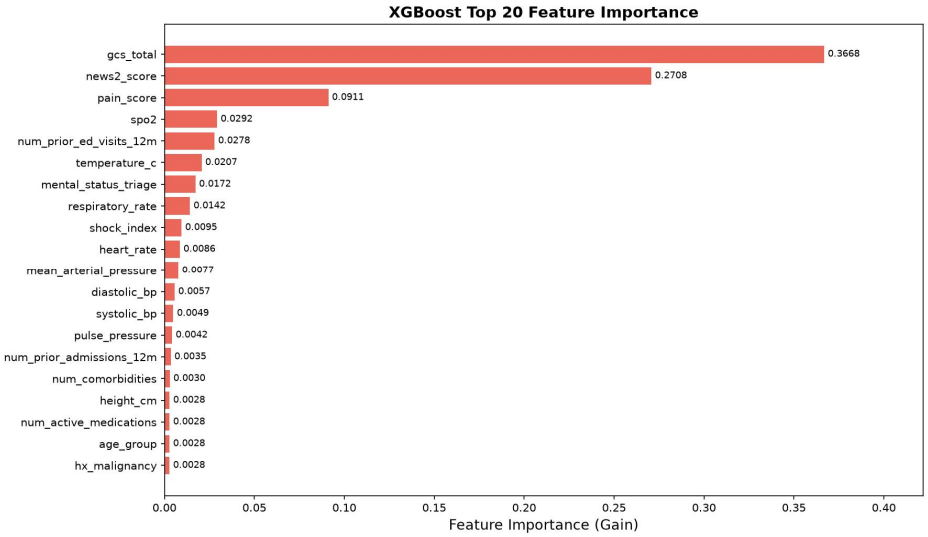


图 7: XGBoost Top 20 特征重要性

NEWS2 评分是最重要的预测特征，这与 NEWS2 与分诊等级高达-0.822 的 Spearman 相关性一致。其他重要的特征包括休克指数、血氧饱和度、心率、GCS 评分等临床核心指标，这从临床角度验证了模型的合理性。

5 讨论

5.1 方法多样性价值

本研究验证了不同建模范式在相同数据上的表现差异。从经典统计学（有序 Logistic 回归）到集成学习（随机森林）再到前沿梯度提升（XGBoost/LightGBM）和深度学习（MLP），覆盖了当代统计建模和机器学习的主流方法。即使共享完全相同的数据预处理流程，不同模型在性能上仍有约 6.4 个百分点的准确率差异（78.89% vs 85.33%），说明了模型选择的重要性。

5.2 临床意义

高 QWK 分数（0.93）表明模型在进行分诊预测时，不仅整体准确率较高，而且严重错误（如将危重患者评估为轻症）的比例极低。XGBoost 模型对 Level 1 患者的识别准确率高达 98.2%，这对于急诊场景中的患者安全至关重要。

5.3 局限性与未来方向

特征文本利用：chief_complaint_raw 原始文本主诉包含丰富信息，但本研究未使用 NLP 方法提取，未来可引入 TF-IDF 或临床 BERT 嵌入；超参数优化：当前使用手动设定参数，进一步使用贝叶斯优化或网格搜索可能带来小幅提升；外部验证：模型仅在单一数据集上评估，其在其他医院/地区的泛化性能（transportability）有待验证（Mejri, 2026）。

6 结论

本研究构建了统一的急诊分诊预测数据处理流水线，并实现了五种不同机器学习模型。XGBoost 以 85.33%准确率和 0.9295 二次加权 Kappa 表现最优，证实了梯度提升方法在临床分诊预测中的领先地位。从有序 Logistic 回归到深度学习的多种方法，为不同场景（高性能预测 vs. 统计推断）提供了灵活选择。本工作验证了机器学习辅助急诊分诊的可行性，为未来临床决策支持系统的开发提供了方法学基础。

参考文献

[1] Raita, Y., Goto, T., Faridi, M. K., Brown, D. F., Camargo, C. A., & Hasegawa, K. (2019). Emergency department triage prediction of clinical outcomes using machine learning models. Critical Care, 23, 64.

[2] Wicha, S., et al. (2025). Machine Learning-Based Model to Classify Emergency Severity Index Levels 1-3 in Febrile Patients With Tachycardia: Thailand Triage Prediction System. Journal of Clinical Medicine Research.

[3] Mejri, A. N. (2026). Multicentre Feasibility and Statistical Validation of Emergency Triage Prediction. GitHub Repository.

[4] Tougma, S., et al. (2025). Emergency Severity Index Protocol with Machine Learning. InterSol 2025, Springer.

[5] Ketseridou, S., et al. (2021). AI-driven prediction for the disposition of medium-risk incidents visiting emergency departments. PCI 2021, ACM.

[6] Kwon, J., et al. (2025). A foundational triage system for improving accuracy in moderate acuity level emergency classifications. Communications Medicine, 5, 322.

附录

附录 A：数据预处理流水线（preprocessing.py）

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import OrdinalEncoder, StandardScaler
from sklearn.impute import SimpleImputer

def load_raw_data():
    """加载原始数据，合并三张表"""
    train = pd.read_csv("data/train.csv")
    test  = pd.read_csv("data/test.csv")
    history    = pd.read_csv("data/patient_history.csv")
    complaints = pd.read_csv("data/chief_complaints.csv")
    complaints_clean = complaints.drop(columns=['chief_complaint_system'], errors='ignore')
    train = train.merge(history, on='patient_id', how='left') \
                 .merge(complaints_clean, on='patient_id', how='left')
    test  = test.merge(history, on='patient_id', how='left') \
                .merge(complaints_clean, on='patient_id', how='left')
    return train, test

def clean_features(train, test):
    """特征清洗：去泄露变量、缺失值填充、特征工程"""
    target   = train['triage_acuity'].copy()
    test_ids = test['patient_id'].copy()
    for df in [train, test]:
        df.drop(columns=['disposition','ed_los_hours','patient_id',
                         'chief_complaint_raw'], errors='ignore', inplace=True)
    encode_cols = [c for c in train.select_dtypes(include=['object']).columns
                   if not c.startswith('hx_')]
    hx_cols  = [c for c in train.columns if c.startswith('hx_')]
    fill_num = [c for c in train.select_dtypes(include=['number']).columns
                if c not in hx_cols]
    imputer = SimpleImputer(strategy='median')
    train[fill_num] = imputer.fit_transform(train[fill_num])
    test[fill_num]  = imputer.transform(test[fill_num])
    for df in [train, test]:
        df['map_abnormal'] = ((df['mean_arterial_pressure'] < 70) |
                              (df['mean_arterial_pressure'] > 100)).astype(int)
        df['shock_high']   = (df['shock_index'] > 0.9).astype(int)
        df['news2_high']   = (df['news2_score'] >= 7).astype(int)
        df['vital_instability'] = ((df['heart_rate'] > 100).astype(int) +
                                   (df['respiratory_rate'] > 20).astype(int) +
                                   df['shock_high'])
    common_cols = [c for c in train.columns if c in test.columns]
    return train[common_cols], test[common_cols], target, test_ids, \
           encode_cols, hx_cols, fill_num

def encode_and_scale(train, test, encode_cols, hx_cols, fill_num):
    """序数编码 + 标准化，返回纯数值特征矩阵"""
    encoder = OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1)
    encoder.fit(pd.concat([train[encode_cols], test[encode_cols]]))
    train_enc = pd.DataFrame(encoder.transform(train[encode_cols]),
```

```
                           columns=encode_cols, index=train.index)
    test_enc  = pd.DataFrame(encoder.transform(test[encode_cols]),
                           columns=encode_cols, index=test.index)
    train_all = pd.concat([train_enc, train[hx_cols].fillna(0),
                          train[fill_num]], axis=1)
    test_all  = pd.concat([test_enc,  test[hx_cols].fillna(0),
                          test[fill_num]],  axis=1)
    test_all = test_all[train_all.columns]
    scaler = StandardScaler()
    num_set = list(set(fill_num) & set(train_all.columns))
    train_all[num_set] = scaler.fit_transform(train_all[num_set])
    test_all[num_set]  = scaler.transform(test_all[num_set])
    return train_all.fillna(0), test_all.fillna(0)

def get_processed_data():
    """一键获取处理好的数据 -> X_train, y_train, X_test, test_ids"""
    train, test = load_raw_data()
    train, test, target, test_ids, enc_cols, hx_cols, fill_num = \
        clean_features(train, test)
    X_train, X_test = encode_and_scale(train, test, enc_cols, hx_cols, fill_num)
    return X_train.values, target.values, X_test.values, test_ids.values
```

附录 B：模型实现代码（models.py，节选）

```
import numpy as np
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, cohen_kappa_score
from sklearn.linear_model import LogisticRegression
from xgboost import XGBClassifier

N_FOLDS = 5

def quadratic_weighted_kappa(y_true, y_pred):
    """二次加权 Kappa——适用于有序分类的评估指标"""
    return cohen_kappa_score(y_true, y_pred, weights='quadratic')

def cv_evaluate(model, X, y, n_folds=N_FOLDS):
    """5 折分层交叉验证，返回 Accuracy 和 QWK 的均值与标准差"""
    skf = StratifiedKFold(n_splits=n_folds, shuffle=True, random_state=42)
    acc_scores, qwk_scores = [], []
    for train_idx, val_idx in skf.split(X, y):
        m = model.__class__(**model.get_params())
        m.fit(X[train_idx], y[train_idx])
        y_pred = m.predict(X[val_idx])
        acc_scores.append(accuracy_score(y[val_idx], y_pred))
        qwk_scores.append(quadratic_weighted_kappa(y[val_idx], y_pred))
    return (np.mean(acc_scores), np.std(acc_scores),
            np.mean(qwk_scores), np.std(qwk_scores))

# ---- 有序 Logistic 回归（经典统计基线）----
def model_ordinal_logistic(X_train, y_train, X_test):
    model = LogisticRegression(solver='lbfgs', max_iter=2000, C=0.5,
                               class_weight='balanced', random_state=42)
    acc, acc_std, qwk, qwk_std = cv_evaluate(model, X_train, y_train)
    model.fit(X_train, y_train)
    return model.predict(X_test), {'accuracy': acc, 'qwk': qwk,
                                   'name': 'Ordinal Logistic Regression'}
```

```
# ---- XGBoost（最佳模型）----
def model_xgboost(X_train, y_train, X_test):
    model = XGBClassifier(
        n_estimators=500, max_depth=8, learning_rate=0.05,
        subsample=0.8, colsample_bytree=0.8, min_child_weight=5,
        reg_alpha=0.1, reg_lambda=1.0, objective='multi:softmax',
        num_class=5, eval_metric='mlogloss', n_jobs=-1,
        random_state=42, verbosity=0)
    y_train_xgb = y_train - 1  # XGBoost 要求 0-based 标签
    acc, acc_std, qwk, qwk_std = cv_evaluate(model, X_train, y_train_xgb)
    model.fit(X_train, y_train_xgb)
    return model.predict(X_test) + 1, {'accuracy': acc, 'qwk': qwk,
                                       'name': 'XGBoost'}
```

附录 C: 主运行脚本（main.py）

```
import numpy as np
import pandas as pd
import os
from preprocessing import get_processed_data
from models import ALL_MODELS

OUTPUT_DIR = "output"
os.makedirs(OUTPUT_DIR, exist_ok=True)

def save_submission(test_ids, predictions, model_name):
    """保存为 Kaggle 提交格式的 CSV"""
    sub = pd.DataFrame({
        'patient_id': test_ids,
        'triage_acuity': predictions.astype(int)
    })
    sub.to_csv(f"{OUTPUT_DIR}/submission_{model_name}.csv", index=False)

def main():
    X_train, y_train, X_test, test_ids = get_processed_data()
    all_scores = {}
    for name, model_fn in ALL_MODELS.items():
        try:
            preds, scores = model_fn(X_train, y_train, X_test)
            all_scores[name] = scores
            save_submission(test_ids, preds, name)
            print(f"  {scores['name']}: Acc={scores['accuracy']:.4f}, "
                  f"QWK={scores['qwk']:.4f}")
        except Exception as e:
            print(f"  ERROR in {name}: {e}")
    best = max(all_scores, key=lambda k: all_scores[k]['qwk'])
    print(f"\n最佳模型: {all_scores[best]['name']} "
          f"(QWK={all_scores[best]['qwk']:.4f})")
    summary = pd.DataFrame([
        {'模型': v['name'], 'Accuracy': f"{v['accuracy']:.4f}",
         'QWK': f"{v['qwk']:.4f}"} for v in all_scores.values()
    ])
    summary.to_csv(f"{OUTPUT_DIR}/model_comparison.csv",
                   index=False, encoding='utf-8-sig')

if __name__ == "__main__":
    main()
```

附录 D：分工安排

姓名	模型任务	其他分工
邢怡博 （组长）	XGBoost （梯度提升， Level-wise）	1. 项目整体统筹与进度协调 2. 主流程脚本整合（main.py） 3. 模型对比框架设计 4. 交叉验证与评估指标实现 5. 课堂汇报（PPT 讲解，12-15 分钟） 6. 代码审查与最终质量把控
程彦铭	有序 Logistic 回归 （经典统计方法）	1. 数据预处理流水线 （缺失值填充、编码、标准化） 2. 统计方法相关文献调研 3. 有序分类方法学部分报告撰写 4. 协助 PPT 中统计方法部分 5. PPT 制作与修改
刘梓恒	随机森林 （Bagging 集成）	1. 探索性数据分析（EDA） + 全部可视化图表制作 2. 集成学习相关文献调研 3. 数据描述章节报告撰写 4. 协助 PPT 中 EDA 可视化部分
陈　林	LightGBM （梯度提升， Leaf-wise）	1. 特征重要性分析与可视化 2. 混淆矩阵生成与分析 3. 附录整理与文档排版 4. Word/PDF 文档格式转换 5. 协助 PPT 中模型对比结果部分
胥超杰	MLP 神经网络 （深度学习）	1. 分析报告全文撰写统筹 （引言、方法、结果、讨论、结论） 2. 深度学习相关文献调研 3. PPT 制作与美化 4. 参考文献整理与格式规范